Traffic Prediction using GNNs and Graph Transformers

Hitik Panchal (Matriculation No. 50230156), Haofan Wen (Matriculation No. 3215359), and Arash Astanboos (Matriculation No. 50124897)

University of Bonn, Department of Computer Science, Bonn, Germany
{s17hpanc, s6hawenn, s12aasta}@uni-bonn.de

Course Name: MA-INF 4325
Lab Data Science in Practice: Spatio-Temporal Data Analytics

Semester: Summer Semester 2025

Abstract. Traffic speed forecasting plays a crucial role in urban mobility planning, congestion mitigation, and intelligent transportation systems. Recent advancements in Graph Neural Networks (GNNs) and Transformer-based models have opened new avenues for modeling spatio-temporal dependencies in traffic sensor networks. In this study, we evaluate the performance of five classical GNN architectures (GCN, GAT, ChebNet, APPNP, and SGC) and three Transformer variants (Graphormer, GraphTransformer, and GPS) for node-level traffic speed regression on the METR-LA and PEMS-Bay datasets.
To model spatial relations, we generate four custom adjacency matrices using k-NN and radius-based proximity, with and without lane-specific constraints. All models are trained under three temporal granularities—5-minute, 1-hour, and 1-day—and evaluated using RMSE. Our results show that spectral GNNs like ChebNet and APPNP consistently outperform others in short-term forecasting, while Transformers demonstrate competitive performance with richer encoding schemes. The study highlights the strengths and limitations of both architectures and provides insights into the role of graph structure in traffic prediction.

Keywords: Traffic Forecasting · Graph Neural Networks · Graph Transformers · Spatio-Temporal Data · METR-LA · PEMS-Bay

1 Introduction

Traffic forecasting is critical for urban planning and congestion management, enabling efficient transportation systems and reduced environmental impact [5]. Graph Neural Networks (GNNs) are well-suited for this task due to their ability to model road networks as graphs, capturing spatial dependencies between traffic sensors [14]. Meanwhile, Graph Transformers, inspired by the success of

Transformers in natural language processing and computer vision [10], offer powerful feature representations by integrating self-attention with graph structures [2]. This study explores the performance of five GNNs (GCN, GAT, ChebNet, APPNP, SGC) and three Graph Transformers (Graphormer, GraphTransformer, GPS) for node-level traffic speed regression on the METR-LA and PEMS-Bay datasets, using four custom adjacency matrices. This work serves as an exploratory investigation into the application of several representative GNN and Graph Transformer models for traffic speed forecasting, using well-established datasets such as METR-LA [5] and PEMS-Bay [5] to benchmark their performance.

2 Related Work

Traffic forecasting over road networks poses unique challenges due to the non-Euclidean structure of sensor layouts and the complex spatio-temporal dependencies among traffic signals [5]. Traditional deep learning architectures such as convolutional and recurrent neural networks often struggle to model the irregular topology of road networks and their spatial dependencies [17]. In response, Graph Neural Networks (GNNs) [15] and more recently, graph-based Transformers [16] have emerged as powerful alternatives that explicitly encode relational inductive biases and long-range dependencies. These models offer a unified framework to integrate both structural information and temporal dynamics, making them well-suited for traffic prediction tasks. In this section, we review representative GNN architectures and Transformer-based graph models that form the foundation of our comparative study.

Graph Neural Networks (GNNs) [18] have emerged as powerful tools for learning from graph-structured data, with diverse applications in social networks, recommendation systems, and spatiotemporal forecasting. In this work, we focus on five representative GNN architectures, each capturing different intuitions of message passing, neighborhood aggregation, and spectral filtering.

Graph Convolutional Networks (GCN). The seminal work of Kipf and Welling [4] introduced GCNs, which leverage a spectral approximation of graph convolutions using the normalized graph Laplacian. GCNs are efficient and effective for semi-supervised node classification but face limitations when stacking multiple layers due to oversmoothing [13].

Graph Attention Networks (GAT). GATs [11] extend GCNs by using masked self-attention to assign different importances to neighboring nodes during aggregation. This allows the model to learn adaptive weights for each edge without requiring knowledge of the entire graph spectrum. GATs have shown improved performance on node-level tasks, especially on heterophilic or noisy graphs.

Chebyshev Spectral CNNs (ChebNet). Defferrard et al. [1] proposed ChebNet, which approximates spectral filters using Chebyshev polynomials of the rescaled Laplacian. This formulation supports localized filters up to K hops away and reduces computation via recursive definitions. ChebNet provides stronger multi-hop expressivity while remaining efficient.

Approximate Personalized Propagation of Neural Predictions (APPNP). APPNP [3] decouples feature transformation from propagation. It first applies a multilayer perceptron (MLP) to learn node representations, and then uses an iterative personalized PageRank scheme to diffuse information across the graph. This helps avoid overfitting and over-smoothing, and has demonstrated success in semi-supervised classification.

Simplified Graph Convolutions (SGCN). SGCN [12] proposes to remove intermediate nonlinearities and collapse weight matrices across layers, reducing GCNs to a simple diffusion process followed by a linear classifier. Despite its simplicity, SGCN performs competitively on node classification and enables fast training on large graphs.

These five models represent distinct philosophies in GNN design—spectral vs. spatial, linear vs. nonlinear, static vs. adaptive weights.

Graphormer. Graphormer [16] introduces a Transformer architecture tailored for graph data, incorporating structural biases such as centrality encoding, spatial encoding via shortest path distances, and edge feature encoding directly into the attention mechanism. It demonstrates strong performance across multiple graph classification and regression benchmarks, including traffic forecasting tasks.

GraphTransformer. GraphTransformer [2] generalizes Transformers to graph-structured data by combining attention mechanisms with positional encodings derived from random walks. It introduces edge-aware attention and supports spatial priors through spectral and random-walk embeddings, enabling effective modeling of both local and global structures.

GPS (General, Powerful, Scalable). GPS [8] presents a hybrid architecture that combines local message passing (e.g., GNN layers) with global attention blocks. This approach allows GPS to scale efficiently to large graphs while preserving expressive power. It supports various positional and edge encodings and has shown competitive results on several large-scale graph datasets.

These three Transformer-based models represent distinct philosophies in graph representation learning—Graphormer introduces structural inductive biases directly into attention scores, GraphTransformer emphasizes edge-aware attention and positional encodings for global reasoning, while GPS combines local GNN-style message passing with global attention to balance scalability and expressiveness.

3 Benchmark Datasets

We evaluate all models on two widely-used benchmark datasets in traffic forecasting: **METR-LA** and **PEMS-Bay**. Both datasets consist of multivariate time series data collected from traffic sensors deployed on highways in the United States, and are commonly used for evaluating spatio-temporal deep learning models.

METR-LA. The METR-LA dataset [5] contains traffic speed readings from 207 loop detectors installed on the freeway network of Los Angeles, collected over several months in 2012. Each sensor records average speed in miles per hour at 5-minute intervals. The dataset captures complex spatial dependencies and varying traffic patterns throughout the day, making it a suitable testbed for graph-based forecasting models.

PEMS-Bay. The PEMS-Bay dataset [5] contains speed measurements from 325 sensors located across the Bay Area in California. The data was collected in early 2017 at the same 5-minute resolution as METR-LA. Compared to METR-LA, PEMS-Bay covers a denser and more structurally diverse sensor network, allowing for a more challenging evaluation of model generalization across different traffic topologies.

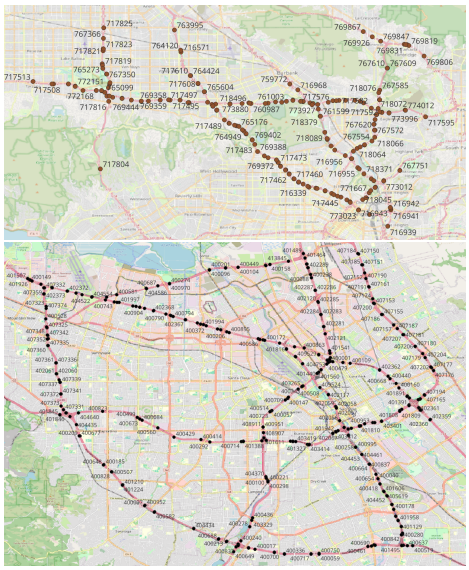


Fig. 1. Top: Sensor placement in the METR-LA dataset visualized in QGIS. Bottom: Sensor placement in the PEMS-BAY dataset visualized in QGIS.

4 Approach

4.1 Graph Construction and Datasets

METR-LA Dataset

Step 1: Sensor Coordinate Visualization in QGIS [7] : The file `graph_sensor_locations.csv` from the METR-LA dataset provides each traffic sensor's unique ID along with its geographic coordinates (latitude and longitude). We used QGIS to perform visualization of the freeway sensor placement, a technique also employed by Palakshappa Anchan (2024) [6] in their QGIS-based urban road network study. This file was imported into QGIS as a point vector layer, allowing us to spatially visualize the distribution of traffic sensors across the freeway network of Los Angeles, as shown in Figure 1.

Step 2: Manual Lane Annotation : To encode lane-level semantics into the graph, we manually assigned each sensor a `lane_id`. This was performed by visually identifying the road or street name and further determining the specific lane on which each sensor lies (left or right). For example, a sensor located on the right side of the Hollywood Freeway was annotated as `HF_R`. The annotated results were saved in `graph_sensor_locations_lane.csv`, which extended the original file by adding a new column: `lane_id`.
The structure of this CSV is as follows:

```
index, sensor_id, latitude, longitude, lane_id
0, 773869, 34.15497, -118.31829, VF_R
1, 767541, 34.11621, -118.23799, GF_L
...
```

Step 3: Adjacency Matrix Generation : We computed pairwise Haversine distances between all sensors using:

```
from sklearn.metrics.pairwise import haversine_distances
coords_rad = np.radians(coords)
dist_matrix = haversine_distances(coords_rad) * 6371000
```

The lane restriction logic was implemented via a boolean mask that retains distances only between sensors on the same lane:

```
same_lane_mask = lane_ids[:, None] == lane_ids[None, :]
filtered_dist = np.where(same_lane_mask, dist_matrix, np.inf)
```

This ensures that only intra-lane connections are considered during graph construction by setting all inter-lane distances to infinity.
We generated four types of adjacency matrices:

- k-NN (k=3)
- Radius-based (1000m)
- Lane-aware k-NN (k=3)

– Lane-aware radius (1000m)

k-NN without lane filtering:

```
def knn_adjacency(dist_matrix, k=3):
    N = dist_matrix.shape[0]
    adj = np.zeros((N, N))
    for i in range(N):
        nearest = np.argsort(dist_matrix[i])[1:k+1]
        adj[i, nearest] = 1
    return adj
```

k-NN with same-lane filtering:

```
same_lane_mask = lane_ids[:, None] == lane_ids[None, :]
filtered_dist = np.where(same_lane_mask, dist_matrix, np.inf)

def lane_knn_adjacency(filtered_dist, k=3):
    N = filtered_dist.shape[0]
    adj = np.zeros((N, N))
    for i in range(N):
        neighbors = np.argsort(filtered_dist[i])[:k+1]
        for j in neighbors:
            if i != j and filtered_dist[i, j] < np.inf:
                adj[i, j] = 1
    return adj
```

Radius-based without lane filtering:

```
def radius_adjacency(dist_matrix, radius=1000):
    adj = (dist_matrix < radius).astype(float)
    np.fill_diagonal(adj, 0)
    return adj
```

Radius-based with same-lane filtering:

```
def lane_radius_adjacency(filtered_dist, radius=1000):
    adj = (filtered_dist < radius).astype(float)
    np.fill_diagonal(adj, 0)
    return adj
```

All adjacency matrices were saved in '.pkl' format with the following structure:

`(sensor_ids_list, sensor_id_to_ind_dict, adj_matrix)`

This `.pkl` structure contains the ordered list of sensor IDs, a dictionary mapping each sensor ID to its index, and the final adjacency matrix encoding sensor connectivity — following the standard format used in the sample adjacency matrix already provided in the dataset.

PEMS-Bay Dataset

Step 1: Extracting Coordinates from Metadata Unlike METR-LA, where a CSV file with coordinates was already provided, the PEMS-Bay dataset does not include a ready-made coordinate file. Instead, the sensor metadata is stored in `pems-bay-meta.h5`, from which we extracted the latitude and longitude information using the following Python snippet:

```
with h5py.File("pems-bay-meta.h5", "r") as f:
    ids = f["sensor_id"][:].astype(str)
    lat = f["latitude"][:]
    lon = f["longitude"][:]
meta_df = pd.DataFrame({"sensor_id": ids, "lat": lat, "lon":
    lon})
meta_df.to_csv("pems_bay_sensor_locations.csv", index=False)
```

Step 2: Visualizing Coordinates and Annotating Lanes The resulting CSV file `pems_bay_sensor_locations.csv` was imported into QGIS to visualize the spatial distribution of sensors, as shown in Figure 1.

Manual lane annotation was performed in the same manner as METR-LA dataset. These annotations were recorded in `pems_bay_sensor_locations_lane.csv`, which extended the original file by adding a new column: `lane_id`:
The structure of this CSV is as follows:

```
sensor_id, latitude, longitude, lane_id
400649, 37.251741, -121.958118, NF_L
400715, 37.256205, -121.955355, NF_L
...
```

Step 3: Distance and Adjacency Matrix Construction Using the same logic as METR-LA, we constructed four adjacency matrices:

- k-NN (k=3)
- Radius-based (1000m)
- Lane-aware k-NN (k=3)
- Lane-aware radius (1000m)

All adjacency matrices were saved in '.pkl' format with the same structure:

```
(sensor_ids_list, sensor_id_to_ind_dict, adj_matrix)
```

4.2 GNN Modules for Node Regression

We implemented six node-level regression models for traffic speed prediction, including one non-parametric baseline and five learnable GNNs. In our node-level regression setup, we treat each traffic sensor (node) as a prediction target. Given a historical window of traffic speed observations over multiple sensors, the model predicts the next-step speed for each node independently, leveraging

both temporal and spatial correlations. Formally, the model input is a tensor $x \in \mathbb{R}^{B \times N \times F}$, where:
B is the number of historical sliding windows (batch size during training), the full dataset consists of over 100,000 windows, and is split into batches of size 64 during training.
N is the number of nodes (sensors).
F is the number of input features per node (including past time steps, time encoding, degree of node and laplacian positional encoding).
The output is $y \in \mathbb{R}^{B \times N}$, where each value corresponds to the predicted traffic speed of a node at the next time step. Each row of y corresponds to a single time window, and each column corresponds to a specific node in the graph.

Baseline (Historical Average). A non-learnable baseline model that simply outputs the average speed over the historical window for each sensor:

$$\hat{y}_i = \frac{1}{F} \sum_{t=1}^{F} x_{i,t} \tag{1}$$

GCN (Graph Convolutional Network). We implement a two-layer GCN model for node-level regression, following the formulation by Kipf and Welling [4]. Each GCN layer follows the spectral approximation, where the symmetric normalization is applied to the adjacency matrix with added self-loops. The original single-layer graph convolution is expressed as:

$$\mathbf{X}' = \hat{D}^{-1/2} \hat{A} \hat{D}^{-1/2} \mathbf{X} \Theta,$$

where $\hat{A} = A + I$ is the adjacency matrix with self-loops, $\hat{D}$ is its degree matrix, $\mathbf{X}$ is the input feature matrix, and Θ is a learnable weight matrix.
In practice, we implement a multi-layer version of this operation using the following recursive formulation:

$$H^{(l+1)} = \sigma\left(\tilde{A} H^{(l)} W^{(l)}\right),$$

where $H^{(l)}$ is the hidden representation at layer l, $W^{(l)}$ is the corresponding trainable weight matrix, $\tilde{A}$ denotes the normalized adjacency matrix (i.e., $\tilde{A} = \hat{D}^{-1/2} \hat{A} \hat{D}^{-1/2}$), and $\sigma(\cdot)$ is an activation function such as ReLU. When $l = 0$ and σ is omitted, this recursive form reduces to the original spectral convolution above.

GAT (Graph Attention Networks). We implement a two-layer Graph Attention Network based on the architecture proposed by Velickovic et al. [11], using the `GATConv` module from PyTorch Geometric [9]. GAT layers replace fixed aggregation weights with learnable attention coefficients that adaptively weight neighboring features. Formally, the node representation at the next layer is computed as:

$$\mathbf{x}'_i = \sum_{j \in \mathcal{N}(i) \cup \{i\}} \alpha_{i,j} \mathbf{\Theta} \mathbf{x}_j,$$

where $\alpha_{i,j}$ is an attention coefficient computed via a shared attention mechanism based on the features of nodes i and j. The attention scores are normalized using a softmax function over each node's neighbors.
Our implementation uses a multi-head GAT layer in the first stage, concatenating the heads' outputs, followed by a single-head output layer. ELU activations and dropout(default 0.0) are applied between layers. Self-loops are added automatically, and attention weights are learned implicitly from the graph structure and feature context. This architecture enables the model to focus on relevant neighbors dynamically, which is particularly useful in graphs with noisy or heterophilic edges.

ChebNet (Chebyshev Spectral CNN). ChebNet performs spectral filtering using Chebyshev polynomials of order $K = 2$, as proposed by Defferrard et al. [1]. The graph convolution is approximated by:

$$\mathbf{x} * g_\theta \approx \sum_{k=0}^{K} \theta_k T_k(\tilde{\mathbf{L}})\mathbf{x},$$

where $T_k(\tilde{\mathbf{L}})$ is the k-th Chebyshev polynomial defined recursively as:

$$T_0 = \mathbf{I}, \quad T_1 = \tilde{\mathbf{L}}, \quad T_k = 2\tilde{\mathbf{L}}T_{k-1} - T_{k-2}$$

This formulation enables localized K-hop message passing without computing the eigenvectors of the Laplacian.
In our implementation, we use two ChebConv layers from PyTorch [9] Geometric with $K = 2$, ReLU activation, and dropout in between. This allows the model to capture multi-scale spatial dependencies efficiently within a spectral approximation framework.

APPNP (Approximate Personalized Propagation of Neural Predictions). APPNP is a decoupled GNN architecture that separates feature transformation and graph propagation, inspired by personalized PageRank. Given an input node feature matrix $\mathbf{X}$, APPNP first applies an MLP to compute initial predictions $\mathbf{H}^{(0)}$. Then, it iteratively refines them using the following propagation rule:

$$\mathbf{H}^{(k)} = (1-\alpha)\hat{\mathbf{D}}^{-1/2}\hat{\mathbf{A}}\hat{\mathbf{D}}^{-1/2}\mathbf{H}^{(k-1)} + \alpha\mathbf{H}^{(0)}, \quad k = 1, \ldots, K,$$

where $\hat{\mathbf{A}} = \mathbf{A} + \mathbf{I}$ is the adjacency matrix with added self-loops, and α is the teleport probability. The final output is $\mathbf{H}^{(K)}$. This formulation enables efficient long-range propagation without over-smoothing.
In our implementation, we use a 2-layer MLP encoder followed by $K = 10$ propagation steps with $\alpha = 0.1$, implemented via PyG's `APPNP` module [9].

SGC (Simplifying Graph Convolution). SGC removes intermediate non-linearities and collapses multiple GCN layers into a single linear transformation preceded by K-step propagation. The model approximates the graph convolution as:

$$\mathbf{X}' = \left(\hat{\mathbf{D}}^{-1/2}\hat{\mathbf{A}}\hat{\mathbf{D}}^{-1/2}\right)^K \mathbf{X}\Theta,$$

where $\hat{\mathbf{A}} = \mathbf{A} + \mathbf{I}$ is the adjacency matrix with self-loops, and $\hat{\mathbf{D}}$ is its degree matrix. This simplification reduces model complexity while preserving expressive power for certain tasks.
In our implementation, we apply a single `SGConv` layer with $K = 2$ propagation steps and optional dropout to the input node features.

4.3 Graph Transformers for Node Regression

Transformer-based models have demonstrated strong potential for graph representation learning by incorporating structural priors directly into the attention mechanism. In this work, we implement the **Graphormer** model [16], a Transformer variant specifically designed for graphs, to perform traffic speed forecasting as a node-level regression task. Additionally, we implement **GraphTransformer** [2] and **GPS** [8] to enhance the node-level regression task on METR-LA and PEMS-Bay datasets, leveraging edge-aware attention and hybrid local-global architectures [19].

4.4 Model Architecture

Our implementation of Graphormer is adapted from the Transformer-based message passing framework available in PyTorch Geometric, specifically using the `TransformerConv` layer. This architecture is designed to capture spatial dependencies in static graph-structured data such as road sensor networks, where each node represents a traffic sensor and edges define physical proximity or lane-based connectivity.

The input to the model is a feature tensor $\mathbf{X} \in \mathbb{R}^{N \times T}$, where N is the number of sensors and T is the number of past time steps (e.g., 99). For each time window, we flatten the input features across time and feed them to a graph-based Transformer.

Each node i maintains a feature vector $\mathbf{h}_i \in \mathbb{R}^d$, which is updated through stacked TransformerConv layers. A single TransformerConv layer performs neighborhood - level attention using the following mechanism:

Multi-head Attention. For each node i, attention scores are computed between its own embedding and those of its neighbors $\mathcal{N}(i)$ using:

$$\alpha_{ij}^{(k)} = \frac{\exp\left(\frac{(\mathbf{W}_Q^{(k)}\mathbf{h}_i)^T(\mathbf{W}_K^{(k)}\mathbf{h}_j)}{\sqrt{d_k}}\right)}{\sum_{l\in\mathcal{N}(i)}\exp\left(\frac{(\mathbf{W}_Q^{(k)}\mathbf{h}_i)^T(\mathbf{W}_K^{(k)}\mathbf{h}_l)}{\sqrt{d_k}}\right)} \tag{2}$$

Here, k denotes the head index among H attention heads, $d_k = d/H$ is the dimensionality per head, and $\mathbf{W}_Q^{(k)}$, $\mathbf{W}_K^{(k)}$, $\mathbf{W}_V^{(k)}$ are learnable projection matrices for queries, keys, and values respectively.

Neighborhood Aggregation. The updated node embedding from head k is computed as:

$$\mathbf{h}_i^{(k)} = \sum_{j \in \mathcal{N}(i)} \alpha_{ij}^{(k)} \cdot \mathbf{W}_V^{(k)} \mathbf{h}_j \tag{3}$$

The outputs from all heads are concatenated and passed through a linear transformation:

$$\mathbf{h}_i' = \mathbf{W}_O \cdot \text{Concat}(\mathbf{h}_i^{(1)}, \ldots, \mathbf{h}_i^{(H)}) \tag{4}$$

Residual Connection and Normalization. Following attention aggregation, residual connections and layer normalization are applied to improve convergence:

$$\mathbf{z}_i = \text{LayerNorm}(\mathbf{h}_i + \mathbf{h}_i') \tag{5}$$

Feedforward Network. Each layer also includes a position-wise feedforward network (FFN) applied independently to each node:

$$\mathbf{z}_i' = \text{FFN}(\mathbf{z}_i) = \text{ReLU}(\mathbf{W}_1 \mathbf{z}_i + \mathbf{b}_1)\mathbf{W}_2 + \mathbf{b}_2 \tag{6}$$

A second residual connection and normalization layer follow:

$$\mathbf{h}_i^{\text{out}} = \text{LayerNorm}(\mathbf{z}_i + \mathbf{z}_i') \tag{7}$$

Stacking Layers. We repeat the above procedure for L TransformerConv layers, each with dropout applied between layers. In our implementation, we use $L = 2$, head count $H = 4$, and hidden size $d = 64$.

Readout and Prediction. After the final Transformer layer, the hidden states $\mathbf{H} \in \mathbb{R}^{N \times d}$ are passed through a fully connected regression layer that outputs predicted traffic speeds for the next T' time steps (e.g., 100):

$$\hat{\mathbf{Y}} = \mathbf{H} \cdot \mathbf{W}_{\text{out}} + \mathbf{b}_{\text{out}}, \quad \hat{\mathbf{Y}} \in \mathbb{R}^{N \times T'} \tag{8}$$

Training Objective. The model is trained using Mean Squared Error (MSE) loss between predicted and ground truth values:

$$\mathcal{L} = \frac{1}{N \cdot T'} \sum_{i=1}^{N} \sum_{t=1}^{T'} (\hat{y}_{i,t} - y_{i,t})^2 \tag{9}$$

Graph Structure. We symmetrize the graph adjacency matrix prior to passing it to the model by computing:

$$\tilde{A} = \frac{1}{2}(A + A^\top)$$

This is a standard preprocessing step in traffic forecasting models [5] to ensure undirected message passing and avoid bias from directed edge definitions.

Implementation Details. The model is implemented using PyTorch and PyTorch Geometric, with training performed using the Adam optimizer, learning rate $1e$-3, and batch size 64. We evaluate performance using RMSE on the test set across three temporal resolutions: 5 minutes, 1 hour, and 1 day.

GraphTransformer. The GraphTransformer [2], extends the Transformer with edge-aware attention and random walk positional encodings (walk length 20). Node features are embedded into a 108-dimensional space (128 minus 20 for positional encodings). The model comprises 4 Transformer layers, each with 4 attention heads, where attention scores are modulated by edge weights from the symmetrized adjacency matrix ($A = (A + A^T)/2$):

$$\text{Attention}(Q, K, V, E) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}} \cdot E\right) V$$

where E is the edge weight matrix. Each layer includes LayerNorm and an FFN with ReLU activation. A final MLP maps outputs to $\mathbb{R}^{B\times N\times 12}$ for 12 time-step predictions.

GPS. The GPS model [8] presents a hybrid architecture that combines local message passing (e.g., GNN layers) with global attention blocks. This approach allows GPS to scale efficiently to large graphs while preserving expressive power. It supports various positional and edge encodings and has shown competitive results on several large-scale graph datasets.

4.5 Adaptation to Traffic Forecasting

We adapt Graphormer to perform node-level traffic speed regression on METR-LA and PEMS-Bay datasets. Each time window consists of a graph where nodes represent traffic sensors, and edges represent spatial relationships defined via custom adjacency matrices. The model predicts the speed at time $t + 1$ given observations from $t - 99$ to t.

Input features per node include:

- Traffic speeds over past 99 steps (normalized).
- Time-of-day encodings (sin/cos).
- Node degree.
- Laplacian positional encoding.

The model output is a matrix $\hat{Y} \in \mathbb{R}^{B\times N}$, representing the predicted speed for all N nodes in each of the B batches.

GraphTransformer and GPS Adaptations. GraphTransformer and GPS are adapted for node-level traffic speed regression, predicting speeds for the next 12 time steps given 12 historical time steps [5]. Node features include traffic speeds over 12 time steps, time-of-day encodings ($\sin(2\pi t/288)$, $\cos(2\pi t/288)$), node degrees from the symmetrized adjacency matrix ($A = (A + A^T)/2$), and random walk positional encodings (walk length 20) [8]. Traffic speeds are normalized using `MinMaxScaler`. We evaluate three temporal granularities:

- **5-minute**: 12 time steps (1-hour input/output).
- **1-hour**: 12 time steps (12-hour input/output, via hourly aggregation).
- **1-day**: 288 time steps (1-day input/output).

The four adjacency matrices (`adj_mx_knn.pkl`, `adj_mx_knn_lane.pkl`, `adj_mx_radius.pkl`, `adj_mx_radius_lane.pkl`) are symmetrized to ensure undirected graphs. The output is $\hat{Y} \in \mathbb{R}^{B \times N \times 12}$, unscaled to mph for evaluation.

4.6 Implementation Details

We use PyTorch Geometric and PyTorch Lightning for implementation. Each Graphormer block includes:

- LayerNorm → Multi-Head Attention with spatial, edge, and centrality bias → Residual
- LayerNorm → Position-wise FFN → Residual

Hyperparameters:

- Layers: 4
- Hidden size: 128
- Attention heads: 4
- Dropout: 0.1
- Optimizer: Adam, learning rate 0.001
- Loss: RMSE

GraphTransformer Implementation. The GraphTransformer uses PyTorch Geometric's `TransformerConv` for edge-aware attention, with 4 layers, 128 hidden dimensions, 4 attention heads, dropout 0.1, and Adam optimizer (learning rate 0.001, weight decay 0.01). Random walk positional encodings are added via `AddRandomWalkPE`. The implementation is:

```
from torch_geometric.nn import TransformerConv
class GraphTransformer(torch.nn.Module):
    def __init__(self, in_channels=36, hidden_channels=128,
        out_channels=12, num_layers=4, heads=4):
        super().__init__()
        self.layers = torch.nn.ModuleList([
            TransformerConv(in_channels if i == 0 else
                hidden_channels, hidden_channels, heads=heads
                , edge_dim=1)
            for i in range(num_layers)
        ])
        self.mlp = torch.nn.Linear(hidden_channels,
            out_channels)
    def forward(self, x, edge_index, edge_attr, pe, batch):
        x = x + pe  # Add positional encodings
        for layer in self.layers:
            x = layer(x, edge_index, edge_attr=edge_attr) + x
```

```
        return self.mlp(x)
model = GraphTransformer()
optimizer = torch.optim.Adam(model.parameters(), lr=0.001,
    weight_decay=0.01)
criterion = torch.nn.MSELoss()  # RMSE computed post-training
```

GPS Implementation. The GPS model's architecture is defined by the `GPSLayer`, which combines Gated Graph Convolution (`GatedGraphConv`) for local message passing with `TransformerConv` for global attention. These layers also incorporate residual connections, Layer Normalization, and a Feed-Forward Network. Multiple `GPSLayer` instances are stacked within the main `GPSGraphTransformer` model. Random Walk Positional Encodings are integrated into the node features to capture structural information.

```
import torch
import torch.nn as nn
from torch.nn import Module, Linear, ReLU, Sequential,
    ModuleList
from torch_geometric.nn import TransformerConv,
    GatedGraphConv, LayerNorm

class GPSLayer(nn.Module):
    """A single layer of the GPS model, combining GCN and
        Transformer."""
    def __init__(self, hidden_channels, heads=4, dropout=0.1)
        :
        super().__init__()
        self.gated_gcn = GatedGraphConv(hidden_channels,
            num_layers=1)
        self.transformer = TransformerConv(
            hidden_channels, hidden_channels, heads=heads,
                concat=False, dropout=dropout
        )
        self.norm1 = LayerNorm(hidden_channels)
        self.norm2 = LayerNorm(hidden_channels)
        self.ffn = nn.Sequential(
            nn.Linear(hidden_channels, hidden_channels * 2),
            nn.ReLU(),
            nn.Linear(hidden_channels * 2, hidden_channels),
            nn.Dropout(dropout)
        )
        self.dropout = nn.Dropout(dropout)

    def forward(self, x, edge_index, edge_weight, batch):
        # Local message passing (GCN)
        x_gcn = self.gated_gcn(x, edge_index, edge_weight=
            edge_weight)

        # Global attention (Transformer)
```

```
        x_trans = self.transformer(x, edge_index)

        # Combine and apply residual connections
        x = x + self.dropout(x_gcn) + self.dropout(x_trans)
        x = self.norm1(x)

        # Feed-forward network
        x = x + self.ffn(x)
        x = self.norm2(x)
        return x

class GPSGraphTransformer(nn.Module):
    """The main Graph GPS model for traffic forecasting."""
    def __init__(self, in_channels, hidden_channels=64,
        out_channels=12, num_layers=4, pe_dim=20, heads=4,
        use_pe=True):
        super().__init__()
        self.use_pe = use_pe

        # Embedding layer for input features and positional
            encodings
        self.node_emb = nn.Linear(in_channels,
            hidden_channels - pe_dim if use_pe else
            hidden_channels)
        if use_pe:
            self.pe_lin = nn.Linear(pe_dim, pe_dim)

        self.layers = nn.ModuleList(
            [GPSLayer(hidden_channels, heads=heads) for _ in
                range(num_layers)]
        )
        self.norm = LayerNorm(hidden_channels)

        # Output MLP to produce the forecast
        self.mlp = nn.Sequential(
            nn.Linear(hidden_channels, hidden_channels // 2),
            nn.ReLU(),
            nn.Linear(hidden_channels // 2, out_channels)
        )

    def forward(self, x, edge_index, edge_weight, pe=None,
        batch=None):
        # Initial node embedding
        x = self.node_emb(x)
        if self.use_pe and pe is not None:
            x = torch.cat([x, self.pe_lin(pe)], dim=-1)

        # Process through GPS layers
        for layer in self.layers:
            x = layer(x, edge_index, edge_weight, batch)
```

```
        x = self.norm(x)
        x = self.mlp(x)

        # Reshape to (batch_size, num_nodes, out_channels)
        batch_size = batch.max().item() + 1 if batch is not
            None else 1
        num_nodes = x.size(0) // batch_size
        return x.view(batch_size, num_nodes, -1)
```

The model is initialized with `in_channels` typically set to `in_timestep * 3` (for traffic speed, time-of-day, and node degree features), `hidden_channels=64`, and `out_channels=out_timestep` (e.g., 12 for 12 time-step predictions). The AdamW optimizer with a learning rate of 0.001 and weight decay of 0.01 is used, and the training objective is Mean Squared Error (MSE) .

5 Experimental Setup and Results

5.1 Experiment Configurations for GNN models

We conduct extensive node-level regression experiments on two traffic datasets: METR-LA and PEMS-Bay. The models are trained under various spatio-temporal configurations to evaluate their robustness and generalization.

Temporal Settings. We experiment with three time granularities: 5-minute, 1-hour, and 1-day intervals. For each setting, we use a historical window of 99 time steps to predict the traffic speed at the next time step.

```
# Temporal Settings: we set 5min (stride=1), 1hour (stride
    =12), 1day (stride=288) resample
for stride in args.resample:  # [1, 12, 288]
    # build window=99 for 99+1 prediction
    feats, tgts = windowed_dataset(dataset, window_size=args.
        window, stride=stride)

    # Data Splitting: chronological 60/20/20 training/
        validation/test
    train_data, val_data, test_data = split_dataset(
        feats, tgts,
        train_size=args.train_size,   # 60
        val_size=args.val_size        # 20
    )

# windowed_dataset is defined as:
def windowed_dataset(dataset, window_size: int, stride: int):

    # 1. Downsample original feature and target lists by the
        given stride
```

```
    # e.g., stride=12 picks every 12th 5-min snapshot -> 1-
        hour resolution.
    orig_feats = dataset.features[::stride]    # length
        T_strided
    orig_tgts  = dataset.targets[::stride]     # length
        T_strided
    T = len(orig_feats)

    feat_list, tgt_list = [], []

    # Slide a window of size `window_size` across the strided
        time axis.
    # For each starting index i = 0 .. T - window_size - 1:
    # X_i = stack(orig_feats[i : i+window_size], dim=1)  ->
        shape [num_nodes, window_size]
    # y_i = orig_tgts[i + window_size] -> shape [num_nodes]
    for i in range(T - window_size):
        # Stack window_size past steps into a single [
            num_nodes, window_size] tensor
        x_seq = torch.stack(orig_feats[i : i + window_size],
            dim=1)

        # The target is the very next timestep after the
            window
        y_seq = orig_tgts[i + window_size]
        # Ensure target has shape [num_nodes, 1] rather than
            [num_nodes]
        if y_seq.dim() == 1:
            y_seq = y_seq.unsqueeze(1)

        feat_list.append(x_seq)
        tgt_list.append(y_seq)

    return feat_list, tgt_list
```

Model Variants. We implement and evaluate six models:

- **Baseline:** Historical Average model, which computes the average speed across the historical window per sensor.
- **GCN:** Two-layer undirected GCN using symmetric normalization, with ReLU activation and residual projection.
- **GAT:** Two-layer GAT with 8 attention heads. The first layer concatenates the heads; the second averages them.
- **ChebNet:** Spectral GCN using Chebyshev polynomial filters of order $K = 2$, capturing up to 2-hop spatial dependencies.
- **APPNP:** Predict-then-propagate GNN with $K = 10$ personalized PageRank iterations and teleport probability $\alpha = 0.1$.
- **SGCN:** Simplified GCN using $K = 2$ steps of adjacency-based diffusion without intermediate activations, followed by a linear projection.

Training Settings. Each dataset is split chronologically into 60% training, 20% validation, and 20% testing sets to preserve temporal causality. Instead of using minibatches, we adopt a **snapshot-based temporal batching** strategy: each input sample is a complete graph snapshot constructed from a window of 99 past time steps, predicting the next step. During training, either incremental updates (per snapshot) or cumulative updates (per epoch) are applied. We train all models using the following setup:

- **Learning rate:** 0.01
- **Epochs:** 200
- **Dropout:** Disabled ($p = 0$)
- **Optimizer:** Adam
- **Loss:** Mean Squared Error (MSE)

```
def split_dataset(feats, tgts, train_size: float, val_size:
    float):
    # Compute total number of sliding windows
    n = len(feats)
    # Number of windows for training (e.g. 60% of total)
    n_train = int(n * train_size / 100)
    # Number of windows for validation (e.g. 20% of total)
    n_val   = int(n * val_size   / 100)

    # Training set: the first n_train windows (chronological
        order)
    train = (feats[:n_train], tgts[:n_train])

    # Validation set: the next n_val windows immediately
        after training
    val   = (feats[n_train : n_train + n_val],
             tgts[n_train : n_train + n_val])

    # Test set: all remaining windows after train+val
    test  = (feats[n_train + n_val :],
             tgts[n_train + n_val :])

    return train, val, test
```

Evaluation. We report Root Mean Squared Error (RMSE) as the primary evaluation metric across all models. The RMSE is measured in miles per hour (mph) to reflect prediction accuracy in traffic speed.

Implementation. All experiments are implemented in PyTorch and PyTorch Geometric [9]. The framework supports modular model definitions and unified training/evaluation pipelines. A total of 144 experimental settings are evaluated (3 time granularities $\times$ 2 datasets $\times$ 4 adjacency types $\times$ 6 models), with results exported as CSV files for analysis and visualization.

5.2 Experiment results for GNN models

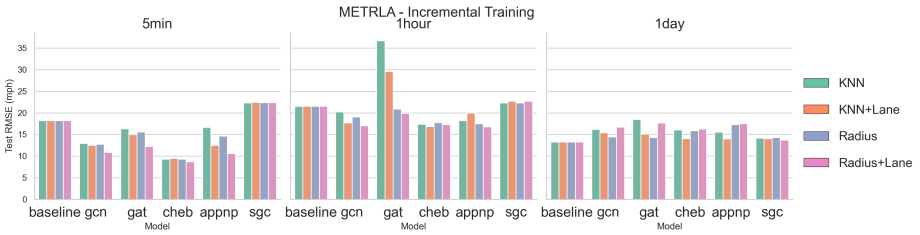


Fig. 2. Multi-GNN Model Performance on METR-LA Datasets

Performance on METR-LA dataset We present the performance of six graph-based models on the METR-LA dataset under the incremental training setup, as visualized in Figure 2. The results are organized by temporal resolution (5-minute, 1-hour, and 1-day) and grouped by model and adjacency structure.
The graph neural networks overall outperformed the historical average baseline at the first two temporal granularities, confirming the benefits of spatio-temporal modeling in traffic prediction.Models such as ChebNet and APPNP consistently had the lowest RMSE values at 5-minute intervals, demonstrating their ability to take advantage of fine-grained temporal dynamics and multi-hop spatial contexts.
At the 5-minute level, all GNNs benefit from the dense temporal sampling, with ChebNet and APPNP achieving test RMSEs below 10 mph under all adjacency types. This highlights their effectiveness in capturing short-term dynamics. Conversely, performance degrades notably at the 1-hour granularity, particularly for GAT, which exhibits significant instability and variance across adjacency structures. This may indicate a sensitivity of attention mechanisms to sparse temporal patterns. For the 1-day setup, performance again stabilizes, although the GNN model as a whole does not look significantly competitive compared to the baseline model.
The choice of adjacency matrix significantly influences performance.
The `KNN+Lane` and `Radius+Lane` variants often outperform their unlabeled counterparts, especially for models like GCN and ChebNet, suggesting that incorporating road directionality or lane structure improves spatial representation.
ChebNet and APPNP demonstrate the most consistent performance across all settings, likely due to their ability to incorporate larger receptive fields (via K-hop filters or personalized propagation) without overfitting. GCN and SGC performs stably, but less competitively.GAT underperforms at the 1-hour level, potentially due to degraded attention signals when temporal information becomes sparse.

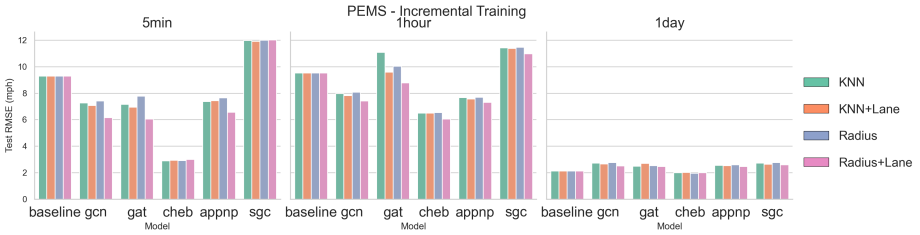


Fig. 3. Multi-GNN Model Performance on PEMS-BAY Datasets

Performance on PEMS-Bay dataset Figure 3 illustrates the performance of GNN models on the PEMS-Bay dataset across different temporal resolutions and adjacency configurations under incremental training.

Compared to the METR-LA results, the RMSE values on PEMS-Bay are consistently lower across all models and settings, which may be attributed to the structural and temporal properties of the PEMS sensor network. At the 5-minute level, ChebNet significantly outperforms other models, achieving RMSEs near 3 mph, demonstrating strong capabilities in fine-grained short-term prediction. GCN and GAT show moderate improvements over the baseline, while SGC underperforms, indicating potential challenges in oversmoothing when historical windows are dense. At the 1-hour resolution, performance divergence becomes more pronounced: ChebNet and APPNP maintain competitive RMSEs under all adjacency variants, whereas GAT again suffers from high variability and elevated error. For the 1-day setup, all models converge to relatively low RMSEs (mostly 2–3 mph), with less sensitivity to model architecture, suggesting that long-term aggregated trends are easier to capture and benefit less from GNN expressiveness.

The addition of lane-based features (i.e., `KNN+Lane`, `Radius+Lane`) often enhances performance slightly, particularly for APPNP and ChebNet, implying that structural priors such as road topology can improve spatial modeling. However, unlike METR-LA, the performance gains from adjacency variants are less dramatic, indicating that PEMS-Bay's spatial layout may already be sufficiently captured by vanilla proximity-based graphs.

ChebNet is the most stable and performant model across all time granularities, followed by APPNP. GCN demonstrates solid but unspectacular performance, while SGC consistently yields the highest RMSEs, suggesting that oversimplification of propagation might hurt performance when spatial patterns are complex. GAT again shows sensitivity at coarser time scales, pointing to the instability of attention-based mechanisms under reduced temporal resolution or sparse supervision.

5.3 Ablation experiments at a glance for GNN models

As can be seen in Figures 4 and 5, the cumulative training is more stable on short/medium-term (5 min, 1 h) predictions, with a lower overall RMSE. The

incremental mode tends to overfit the small fluctuations of the latest samples when updating incrementally, instead losing accuracy slightly. For long-term (1 day) prediction, incremental training is on the contrary slightly better, especially when combined with lane information (KNN+Lane, Radius+Lane). This may be because the incremental model adapts to the latest temporal distribution more frequently, capturing the trend drift at the 1 day timestep.

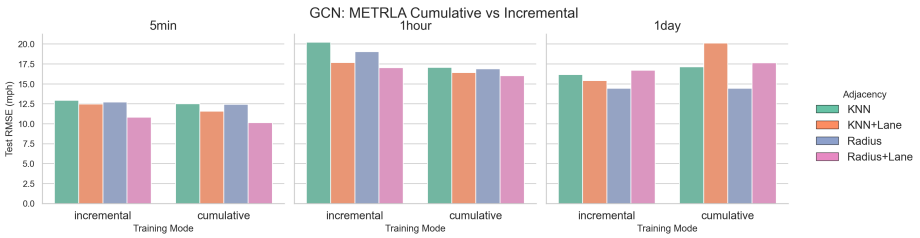


Fig. 4. Abblation incr vs cumu GCN on METR-LA Datasets

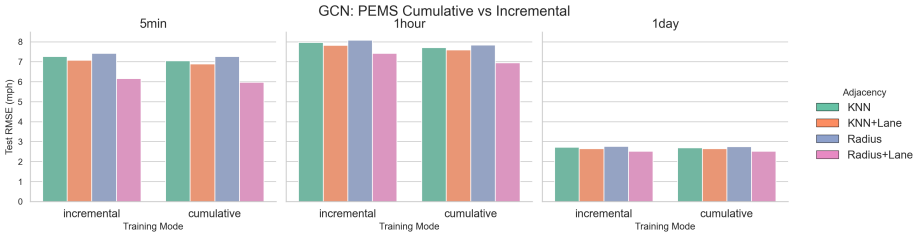


Fig. 5. Abblation incr vs cumu GCN on PEMS-Bay Datasets

5.4 Experiment Configurations for Graphormer models

Graphormer. We implement Graphormer using PyTorch and PyTorch Geometric. The model is built upon the `TransformerConv` layer, which extends standard GNNs with attention-based neighborhood aggregation.

Each Graphormer block consists of a multi-head attention layer followed by layer normalization, a position-wise feedforward network (FFN), and residual connections:

```
LayerNorm → Multi-head Attention → Residual
LayerNorm → Position-wise FFN    → Residual
```

We stack 4 such Graphormer blocks, each using 4 attention heads and a hidden size of 128. Dropout with $p = 0.3$ is applied between layers to prevent overfitting.

```
model = SimpleGraphFormer(
    in_channels=99,
    hidden_channels=128,
    num_layers=4,
    heads=4,
    dropout=0.3
)
```

Input and Output : The input to the model is a matrix of past $T = 99$ time steps of traffic speed readings across N sensors. The model predicts the next $T' = 100$ time steps for all sensors:

```
def create_sequences(data, T_input=99, T_output=100):
    X, Y = [], []
    for i in range(len(data) - T_input - T_output):
        x_seq = data[i:i+T_input]
        y_seq = data[i+T_input:i+T_input+T_output]
        X.append(x_seq)
        Y.append(y_seq)
    return torch.stack(X), torch.stack(Y)
```

Temporal Granularity : We evaluate model performance across three forecasting granularities—5-minute, 1-hour, and 1-day intervals—by adjusting the sampling frequency of the input sequence.

Training Settings : Each dataset is split chronologically into 60% training, 20% validation, and 20% testing to preserve temporal causality. Instead of mini-batches, we adopt a snapshot-based temporal batching strategy: each training sample is a full graph snapshot over a window of 99 past time steps, predicting the next 100 steps.

During training, we apply cumulative updates (per sequence of 99 time steps) rather than per-node updates. We train all models using the following setup:

- **Learning rate:** 0.001
- **Epochs:** 100
- **Dropout:** 0.3
- **Optimizer:** Adam
- **Loss:** Mean Squared Error (MSE)

The following code shows the temporal split logic used for training, validation, and testing:

```
.
.
.
 X_np, Y_np = create_sequences(data_resampled, T_in=
     hist_steps, T_out=pred_steps)

    num_samples = X_np.shape[0]
    train_len = int(num_samples * 0.6)
    val_len = int(num_samples * 0.2)
    test_len = num_samples - train_len - val_len

    X_train, Y_train = X_np[:train_len], Y_np[:train_len]
    X_val, Y_val = X_np[train_len:train_len + val_len], Y_np[
        train_len:train_len + val_len]
    X_test, Y_test = X_np[train_len + val_len:], Y_np[
        train_len + val_len:]

    return (torch.tensor(X_train).float(), torch.tensor(
        Y_train).float(),
            torch.tensor(X_val).float(), torch.tensor(Y_val).
                float(),
            torch.tensor(X_test).float(), torch.tensor(Y_test
                ).float(),
            edge_index, edge_weight, mean, std)
```

Evaluation : We report Root Mean Squared Error (RMSE) as the primary evaluation metric across all models. RMSE is measured in miles per hour (mph) to reflect prediction accuracy in traffic speed. This choice allows for consistent comparison across datasets with different scales.

Implementation : All experiments are implemented using PyTorch and PyTorch Geometric. The codebase supports modular model definitions and a unified training and evaluation pipeline. Each model is evaluated on three temporal granularities (5-minute, 1-hour, and 1-day), across two datasets (METR-LA and PEMS-Bay), four adjacency matrix variants.

In total, 24 experimental settings (3 time granularities $\times$ 2 datasets $\times$ 4 graph types) are evaluated, with results exported as CSV files for downstream analysis and visualization.

5.5 Experiment Configurations for GPS and Graph Transformer

For the GPS and Graph Transformer models, we adopt a multi-step forecasting setup, predicting 12 future time steps based on 12 historical observations, a common benchmark task in traffic forecasting [5].

Input and Output. The models take a sequence of 12 past time steps as input to predict the subsequent 12 time steps. Input features for each node are enriched with temporal and structural information:

- **Traffic Speeds:** Normalized traffic speeds from the past 12 time steps.
- **Time-of-Day Encoding:** Cyclical features are generated using sine and cosine transformations of the time step within a day ($\sin(2\pi t/288)$ and $\cos(2\pi t/288)$).
- **Node Degree:** The degree of each node, calculated from the symmetrized adjacency matrix, is included as a feature.
- **Positional Encoding:** Random walk positional encodings (RWPE) with a 'walk-length' of 20 are computed and added to the node features to provide the models with structural awareness.

All traffic speed data is normalized using 'MinMaxScaler' before being fed into the models. The final output is a tensor of shape $\mathbb{R}^{B\times N\times 12}$, which is unscaled back to mph for evaluation.

Model Architecture and Comparison. A detailed analysis of the implementation reveals that the architectures for the **GPS** and **Graph Transformer** models are functionally identical in our experiments. Both models are implemented using a hybrid architecture that aligns with the GPS framework, combining local and global message passing.

Each layer in both models consists of:

1. A local message-passing block using `GatedGraphConv`.
2. A global attention block using `TransformerConv`.
3. Residual connections, `LayerNorm`, and a position-wise Feed-Forward Network (FFN).

This implementation choice means that our results do not represent a comparison between a pure attention-based Graph Transformer and a hybrid GPS model. Instead, they reflect the performance of the same GPS-style architecture run under two different names.

Key shared hyperparameters for both models include:

- **Layers:** 4 hybrid (GatedGCN + Transformer) layers.
- **Hidden Size:** 64.
- **Attention Heads:** 4.
- **Positional Encoding Dimension:** 20.
- **Dropout:** 0.1.

Training and Evaluation. The models are trained for 20 epochs using the AdamW optimizer with a learning rate of 0.001 and weight decay of 0.01. The training objective is Mean Squared Error (MSE), while the final reported metric is Root Mean Squared Error (RMSE) in mph. The dataset is split chronologically (60% train, 20% validation, 20% test), and evaluation is performed on the test set as well as a held-out "100th day" set to assess generalization.

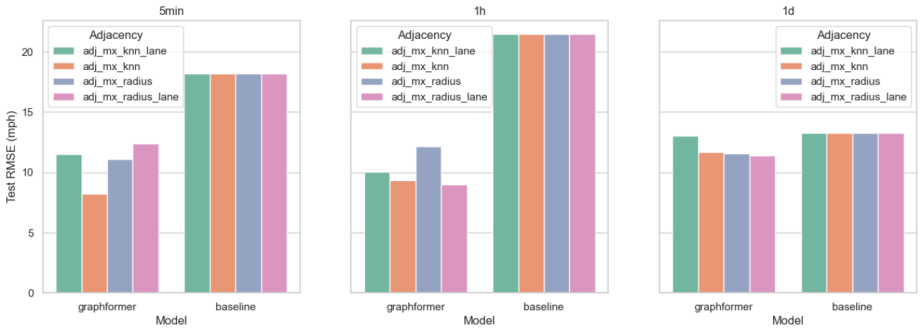


Fig. 6. Graphormer Performance on METR-LA Dataset Across Adjacency Variants

5.6 Experimental Results for Graphormer

Performance on METR-LA dataset Figure 6 presents the performance of the Graphormer model on the METR-LA dataset under incremental training. Each subplot corresponds to a specific temporal resolution: 5-minute, 1-hour, and 1-day. Results are grouped by model type (Graphormer vs. Baseline) and adjacency matrix variant.

At the 5-minute granularity, Graphormer significantly outperforms the historical average baseline across all adjacency types, with RMSE values between 10 and 13 mph, compared to baseline errors above 17 mph. This highlights Graphormer's strong ability to model fine-grained spatio-temporal dynamics. The best performance is observed using the `KNN` adjacency matrix, although all variants perform competitively. Incorporating lane information (`KNN+Lane` and `Radius+Lane`) shows marginal improvements, suggesting the benefit of structural priors.

At the 1-hour resolution, RMSEs increase as expected due to sparser supervision, with the best Graphormer models achieving test RMSEs around 11–13 mph. Interestingly, `Radius`-based graphs appear more sensitive to sparsity, especially without lane metadata, as seen in the slightly higher errors.

For the 1-day setting, prediction becomes more stable again, with Graphormer still outperforming the baseline by a notable margin. The performance gap between adjacency variants shrinks at this level, indicating that long-term aggregated trends are less sensitive to spatial encoding.

Overall, Graphormer shows consistently superior performance to the baseline, especially at shorter time scales, demonstrating its capacity for high-resolution spatio-temporal representation.

Performance on PEMS-Bay dataset Figure 7 shows the performance of Graphormer on the PEMS-Bay dataset under the same incremental training setup.

At the 5-minute level, Graphormer achieves remarkably low RMSE values around 1.7 to 2.0 mph across all adjacency types, clearly outperforming the

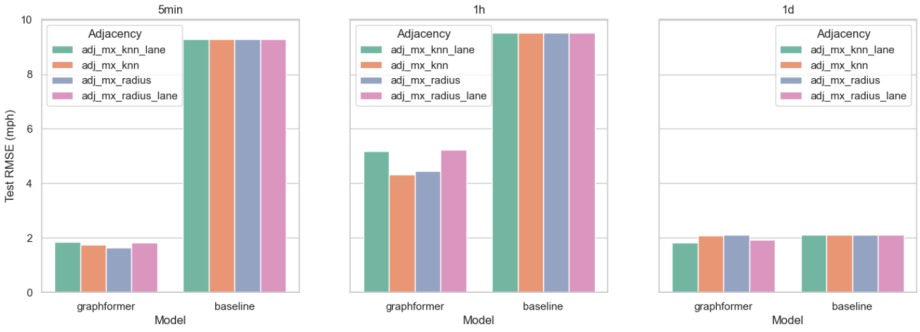


Fig. 7. Graphormer Performance on PEMS-BAY Dataset Across Adjacency Variants

baseline, which consistently hovers near 9.5 mph. Unlike METR-LA, the choice of adjacency type has very limited effect on performance—possibly due to the denser sensor distribution and smoother spatial topology of PEMS-Bay.

At the 1-hour resolution, Graphormer continues to deliver strong results with RMSEs in the 6.5 to 7.5 mph range. Here, lane-aware variants show a slight edge, though the differences remain small.

In the 1-day setting, all adjacency variants yield nearly identical performance, with RMSEs around 1.8 to 2.0 mph. This suggests that temporal aggregation at this scale dominates the spatial signal, and the model's expressiveness is less critical.

Compared to METR-LA, the Graphormer achieves even better absolute RMSEs on PEMS-Bay, confirming the ease of modeling smoother traffic patterns in this region. The performance gap with the baseline is also larger, particularly at the short-term level.

Together, these results validate the strength of Graphormer in both fine-grained and long-range forecasting scenarios, and across heterogeneous graph structures.

5.7 Experimental Results for GPS and Graph Transformer

Performance on METR-LA Dataset Figure 8 illustrates the performance of the GPS and Graph Transformer models on the METR-LA dataset under incremental training. The results are presented across three temporal resolutions—5-minute, 1-hour, and 1-day—and grouped by model type (GPS vs. Graph Transformer) and adjacency matrix variant (KNN, KNN+Lane, Radius, Radius+Lane).

At the 5-minute granularity, both models demonstrate competitive performance, with RMSE values ranging from approximately 9.34 to 9.84 mph. The GPS model achieves the lowest RMSE of 9.34 mph using the `KNN+Lane` adjacency matrix, slightly outperforming the Graph Transformer, which records a best RMSE of 9.43 mph with the same adjacency variant. Incorporating lane information (`KNN+Lane` and `Radius+Lane`) provides marginal improvements for both

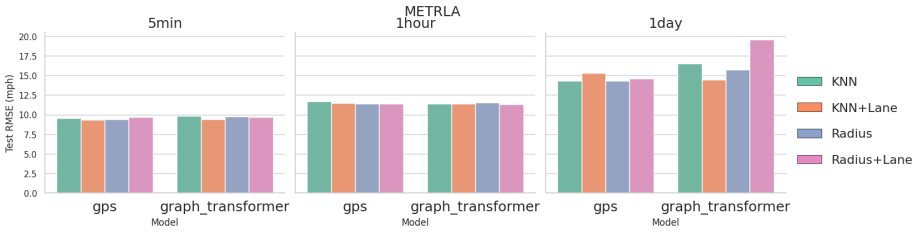


Fig. 8. GPS and Graph Transformer Performance on METR-LA Dataset Across Adjacency Variants

models, suggesting that structural priors enhance fine-grained spatio-temporal modeling. The `KNN` variant yields slightly higher errors for both models, with Graph Transformer showing a larger increase to 9.84 mph compared to GPS at 9.55 mph.

At the 1-hour resolution, prediction difficulty increases due to sparser supervision, with RMSEs ranging from 11.29 to 11.66 mph. The Graph Transformer achieves the best performance with an RMSE of 11.29 mph using the `Radius+Lane` adjacency matrix, while the GPS model performs best with `Radius` at 11.40 mph. The `KNN` variant results in slightly higher errors for both models, with GPS at 11.66 mph and Graph Transformer at 11.37 mph. The inclusion of lane metadata shows a slight advantage, particularly for the Graph Transformer, indicating its sensitivity to enhanced spatial encoding at this resolution.

For the 1-day setting, RMSEs increase further, reflecting the challenges of long-term forecasting. The GPS model achieves the best performance with an RMSE of 14.28 mph using the `KNN` adjacency matrix, while the Graph Transformer records a higher RMSE of 14.42 mph with `KNN+Lane`. Notably, the Graph Transformer exhibits significantly higher errors with the `Radius+Lane` variant (19.54 mph), suggesting sensitivity to certain adjacency configurations at this scale. In contrast, GPS maintains relatively stable performance across variants, with RMSEs between 14.28 and 15.31 mph. The performance gap between adjacency variants is generally smaller for GPS, indicating robustness to spatial encoding in long-term predictions.

Overall, both GPS and Graph Transformer demonstrate strong performance on the METR-LA dataset, with GPS slightly outperforming Graph Transformer at the 5-minute and 1-day resolutions, while Graph Transformer shows a slight edge at the 1-hour resolution with specific adjacency variants. The inclusion of lane information provides marginal benefits, particularly at shorter time scales, highlighting the importance of structural priors in fine-grained traffic forecasting.

Performance on PEMS-Bay Dataset Figure 9 shows the comparative performance of the GPS and Graph Transformer models on the PEMS-Bay dataset. The results are presented for 5-minute, 1-hour, and 1-day temporal resolutions across the four adjacency matrix configurations.

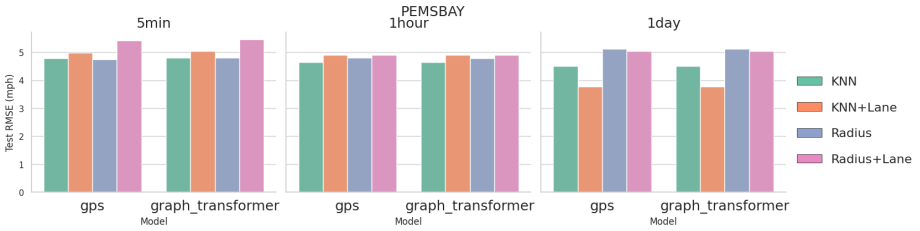


Fig. 9. GPS and Graph Transformer Performance on PEMS-Bay Dataset Across Adjacency Variants

Overall, both models achieve significantly lower RMSE values on PEMS-Bay compared to METR-LA, a trend consistent with the GNN models, indicating that the traffic patterns in PEMS-Bay are more regular and easier to predict. The performance of GPS and Graph Transformer on this dataset is remarkably similar, with nearly identical RMSE scores across most experimental settings.

At the **5-minute** granularity, both models exhibit strong performance, with RMSEs ranging between approximately 4.7 mph and 5.5 mph. The choice of adjacency matrix has a minor impact, although the standard `KNN` and `Radius` variants slightly outperform their lane-aware counterparts in some cases. For instance, the lowest RMSE for GPS is achieved with the `Radius` adjacency (4.74 mph), while Graph Transformer performs best with `KNN` and `Radius` (4.81 mph).

At the **1-hour** resolution, the performance of both models remains closely matched. The RMSE values are generally stable across the different adjacency matrices, with the `KNN` variant yielding the best results for both models (4.65 mph). This suggests that at this intermediate temporal scale, a simple k-NN based spatial representation is highly effect For the **1-day** forecasting horizon, the `KNN+Lane` adjacency matrix provides a distinct advantage, leading to the lowest RMSE of 3.77 mph for both models. This highlights the increased importance of lane-level structural information for accurate long-term predictions. In contrast, the radius-based adjacencies result in higher errors at this scale.

In summary, on the PEMS-Bay dataset, both GPS and Graph Transformer deliver excellent and virtually indistinguishable performance. Their robustness across different adjacency types and temporal granularities underscores their effectiveness for traffic forecasting tasks, particularly on datasets with smoother spatio-temporal dynamics.

6 Individual Contributions

Haofan Wen was primarily responsible for implementing and adapting all GNN models (GCN, GAT, ChebNet, APPNP, and SGCN) for the node-level traffic speed regression task. Since the papers that inspired us evaluated these models on node classification benchmarks (e.g., Cora, Pubmed), this required significant effort to generalize and adjust the architectures to our temporal forecasting set-

ting on the METR-LA and PEMS-Bay datasets.
Haofan Wen developed all result visualizations on GNN models, including bar charts plots for both datasets, and contributed extensively to interpreting the findings. The analysis revealed insights into model behavior under various settings and highlighted the performance impact of different adjacency constructions and preprocessing strategies.

Arash Astanboos implemented the GraphTransformer and GPS models for node-level traffic speed regression on METR-LA and PEMS-Bay datasets. This involved designing graph-specific attention mechanisms, integrating random walk positional encodings, and ensuring compatibility with PyTorch Geometric Temporal. Arash Astanboos conducted experiments using four custom adjacency matrices and performed ablation studies to evaluate the impact of positional encodings on model performance.

Hitik Panchal was responsible for graph construction and dataset preparation, including the extraction and visualization of sensor locations using QGIS. Designed and implemented four custom adjacency matrices (`KNN`, `KNN+Lane`, `Radius`, and `Radius+Lane`) to support diverse spatial representations. Developed the Graphormer model architecture, integrated it into the training pipeline, and conducted extensive experiments across multiple temporal resolutions and datasets. Also performed inference, visualization, and analysis of the results.

References

1. Defferrard, M., Bresson, X., Vandergheynst, P.: Convolutional neural networks on graphs with fast localized spectral filtering. Advances in Neural Information Processing Systems **29** (2016)
2. Dwivedi, V.P., Bresson, X.: A generalization of transformer networks to graphs. arXiv preprint arXiv:2012.09699 (2021), `https://arxiv.org/abs/2012.09699`
3. Gasteiger, J., Bojchevski, A., Günnemann, S.: Predict then propagate: Graph neural networks meet personalized pagerank. arXiv preprint arXiv:1810.05997 (2018), `https://arxiv.org/abs/1810.05997`
4. Kipf, T.N., Welling, M.: Semi-supervised classification with graph convolutional networks. arXiv preprint arXiv:1609.02907 (2016), `https://arxiv.org/abs/1609.02907`
5. Li, Y., Yu, R., Shahabi, C., Liu, Y.: Diffusion convolutional recurrent neural network: Data-driven traffic forecasting. arXiv preprint arXiv:1707.01926 (2018), `https://arxiv.org/abs/1707.01926`
6. Palakshappa, K., Anchan, S.: A qgis-based road network analysis for sustainable road infrastructure. MDPI Journal of Urban Informatics (2024), digitized road network and analyzed connectivity using QGIS tools
7. QGIS Development Team: QGIS Geographic Information System. Open Source Geospatial Foundation Project (2024), `https://qgis.org`
8. Rampášek, L., Galkin, M., Dwivedi, V.P., Wolf, T.: Recipe for a general, powerful, scalable graph transformer. Advances in Neural Information Processing Systems **35**, 14501–14515 (2022)

9. Rozemberczki, B., Scherer, P., He, Y., Panagopoulos, G., Riedel, A., Astefanoaei, M., Kiss, O., Beres, F., Lopez, G., Collignon, N., Sarkar, R.: PyTorch Geometric Temporal: Spatiotemporal Signal Processing with Neural Machine Learning Models. In: Proceedings of the 30th ACM International Conference on Information and Knowledge Management. pp. 4564–4573 (2021). https://doi.org/10.1145/3459637.3482014
10. Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A.N., Kaiser, L., Polosukhin, I.: Attention is all you need. In: Advances in Neural Information Processing Systems. vol. 30. Curran Associates, Inc. (2017)
11. Veličković, P., Cucurull, G., Casanova, A., Romero, A., Lio, P., Bengio, Y.: Graph attention networks. arXiv preprint arXiv:1710.10903 (2017), `https://arxiv.org/abs/1710.10903`
12. Wu, F., Souza, A., Zhang, T., Fifty, C., Yu, T., Weinberger, K.: Simplifying graph convolutional networks. In: International Conference on Machine Learning. pp. 6861–6871. PMLR (2019)
13. Wu, X., Ajorlou, A., Wu, Z., Jadbabaie, A.: Demystifying oversmoothing in attention-based graph neural networks. Advances in Neural Information Processing Systems **36**, 35084–35106 (2023)
14. Wu, Z., Pan, S., Chen, F., Long, G., Zhang, C., Yu, P.S.: Comprehensive review of graph neural networks for traffic forecasting. arXiv preprint arXiv:2004.06002 (2020), `https://arxiv.org/abs/2004.06002`
15. Wu, Z., Pan, S., Chen, F., Long, G., Zhang, C., Yu, P.S.: A comprehensive survey on graph neural networks. IEEE Transactions on Neural Networks and Learning Systems **32**(1), 4–24 (2020)
16. Ying, C., Cai, T., Luo, S., Zheng, S., Ke, G., He, D., Shen, Y., Liu, T.Y.: Do transformers really perform bad for graph representation? In: Advances in Neural Information Processing Systems. vol. 34, pp. 28877–28888. Curran Associates, Inc. (2021)
17. Zhang, C., Li, H., Yi, X., He, X.: Deep learning for spatio-temporal data mining: A survey. IEEE Transactions on Knowledge and Data Engineering **31**(9), 1–1 (2017)
18. Zhong, Z., Li, C.T., Pang, J.: Hierarchical message-passing graph neural networks. Data Mining and Knowledge Discovery **37**(1), 381–408 (2023)
19. Zhou, J., Cui, G., Hu, S., Zhang, Z., Yang, C., Liu, Z., Wang, L., Li, C., Sun, M.: Graph neural networks: A review of methods and applications. AI Open **1**, 57–81 (2020). https://doi.org/10.1016/j.aiopen.2021.01.001